



Машинное обучение

Лекция № 13

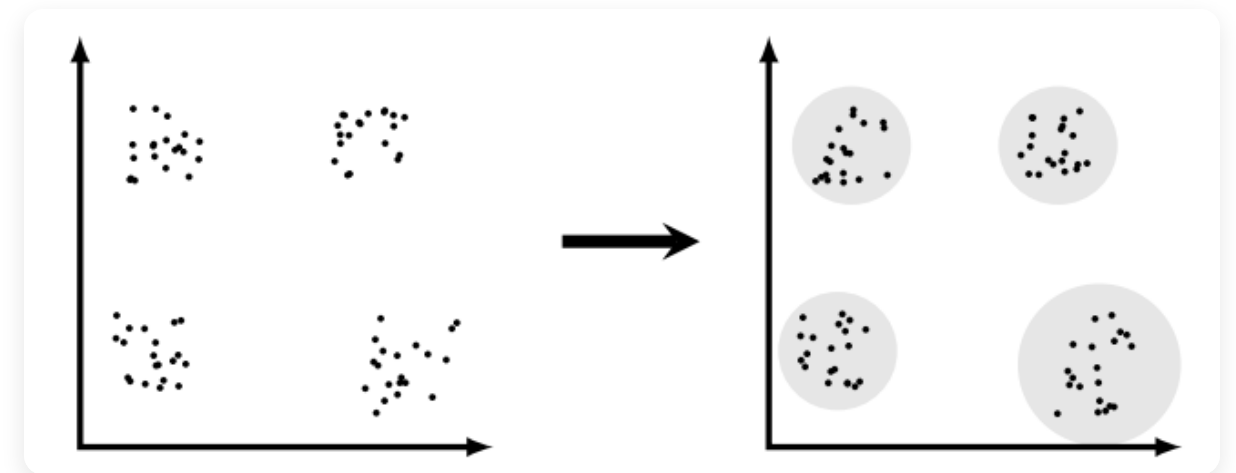
Кластеризация

Докладчик: Ваня Горохов



Обучение без учителя

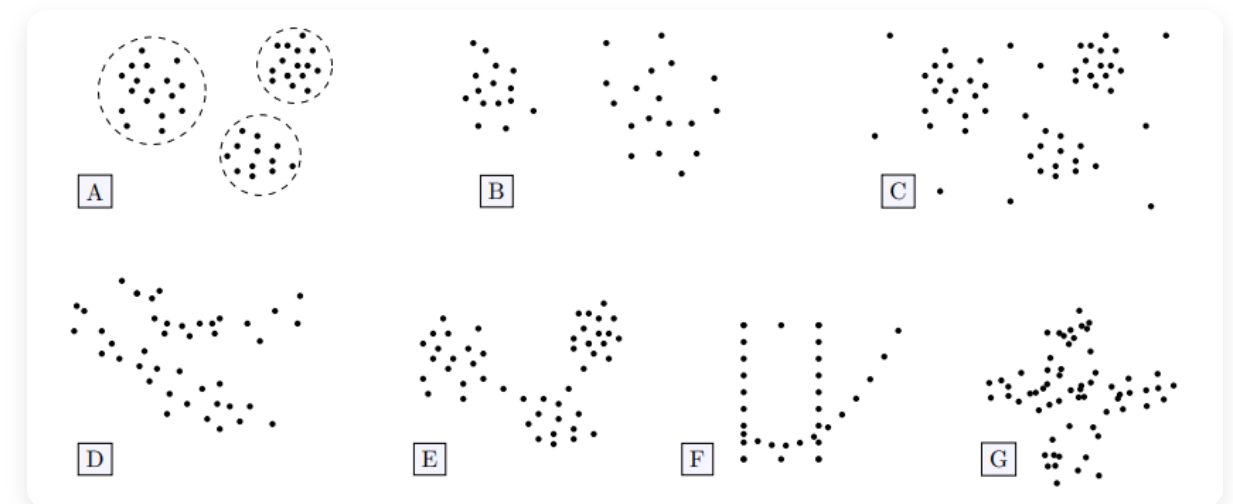
- **Обучение с учителем:** есть признаковое описание X и правильные ответы Y . Задача - минимизировать функцию потерь.
- **Обучение без учителя:** есть только множество объектов X .
- **Задача кластеризации:** разбить объекты на непересекающиеся подмножества (кластеры) так, чтобы объекты из одного кластера были максимально похожи, а из разных - минимально.





Особенности задачи

- **Формы кластеров:** даже в двумерном пространстве формы кластеров могут быть необычными. В многомерном все становится ещё сложнее.
- **Конечная цель задачи:**
 - *Основная:* Результат нужен сам по себе.
 - *Вспомогательная:* Кластеры используются как новые признаки для решения другой задачи.





Метод K-Means

Один из самых популярных методов кластеризации, в котором **число кластеров K является гиперпараметром.**

- Алгоритм пытается найти центры кластеров и привязать каждую точку к ближайшему центру.
- Доказана сходимость за конечное число итераций, из-за этого алгоритм часто работает быстро.

Шаги алгоритма:

1. Выбираются k случайных точек, которые становятся начальными центроидами.
2. Каждая точка выборки относится к тому кластеру, расстояние до центра которого минимально.
3. В качестве нового центра кластера выбирается среднее арифметическое всех точек, попавших в этот кластер.
4. Шаги 2 и 3 повторяются до достижения удовлетворительного результата.



А какой результат удовлетворительный?

- **Max_iter:** Сами задаем, сколько итераций выполнить.
- **Tolerance:** Смотрим, как двигаются центры на каждом шаге. Если сумма квадратов расстояний от старых центров до новых стала меньше заданного порога, останавливаем итерации.
- **Строгая сходимость:** Алгоритм прекращает работу, если ни одна точка не поменяла свой кластер.

```
from sklearn.cluster import KMeans

kmeans = KMeans(
    n_clusters=3,
    max_iter=300, # Max_iter
    tol=1e-4      # Tolerance
)
kmeans.fit(X)
```



Проблема инициализации

Случайный выбор начальных центров может привести к тому, что они попадут в пустые области, либо несколько центров окажутся слишком кучно в одном кластере.

Эвристика K-Means++ позволяет брать максимально удаленные друг от друга центры:

- Первый центр выбираем *равномерно случайно* из точек выборки.
- Для каждой точки вычисляем квадрат расстояния до ближайшего выбранного центра.
- Каждый следующий центр выбираем из точек выборки с вероятностью, пропорциональной этому квадрату расстояния.



Mini Batch K-Means

Проблема: обновление центроидов по всей выборке имеет сложность $O(N)$. В ситуации, когда количество объектов велико, итерации будут долгими.

- **Идея:** На каждой итерации считать новые положения центров не по всем данным, а по небольшой случайной подвыборке.
- **Плюсы:** Алгоритм работает в разы быстрее и требует значительно меньше оперативной памяти.
- **Минусы:** Траектория движения центроидов получается "шумной". Итоговое качество может быть хуже, чем у классического алгоритма.



Bisect K-Means

Что делать, если мы не знаем оптимальное число кластеров K , или хотим получить иерархическую структуру?

Алгоритм разделяющих К-средних:

1. Вначале вся выборка считается одним большим кластером.
2. Применяется классический алгоритм 2-Means.
3. Среди существующих кластеров выбирается один для дальнейшего дробления (тот, у которого максимальная внутрикластерная дисперсия).
4. Для выбранного кластера снова запускается 2-Means.
5. Процесс продолжается до достижения заданного K или удовлетворения критерия качества.

Подбор K : Мы можем останавливать дробление, когда качество кластеризации перестает значительно улучшаться, что позволяет эффективно подбирать значение K .



Насколько хорош K-Means?

K-Means совершенно по-разному работает в зависимости от формы кластеров.

- Алгоритм может решить, что два вытянутых кластера — это одно скопление.
- Если данные имеют форму полумесяцев или вложенных колец — метод не сможет их разделить.
- В случае неодинакового разброса алгоритм некорректно проводит границу.



Крестиками отмечены центроиды. Алгоритм неспособен выделить невыпуклые структуры.



Почему так происходит?

Все дело в функционале ошибки, который минимизирует K-Means — **внутрикластерном расстоянии**:

$$L(C, \mu) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \rightarrow \min_{C, \mu}$$

- Алгоритм штрафует за квадрат расстояния до центроида.
- Это заставляет метод искать **выпуклые, шарообразные** сгустки точек примерно одинакового радиуса.
- Точки, находящиеся на концах длинного кластера, могут оказаться линейно дальше от своего центра, чем от центра соседнего кластера. Алгоритм просто "отрежет" их.

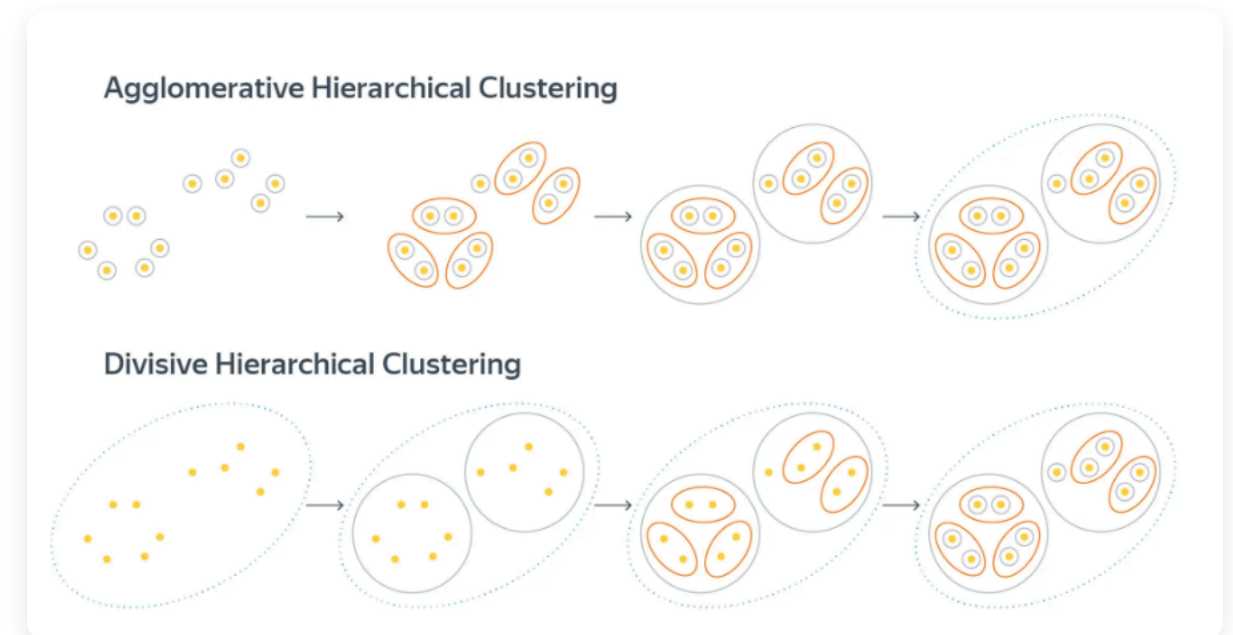


Иерархическая кластеризация

Альтернативный подход: кластеры объединяются или разделяются постепенно, образуя иерархию (дерево).

Агломеративный подход: В начале каждый объект — отдельный кластер. На каждом шаге два самых близких кластера объединяются в один.

Дивизивный подход: Все объекты в одном кластере. На каждом шаге самый большой кластер делится на два, пока каждый объект не станет отдельным кластером.

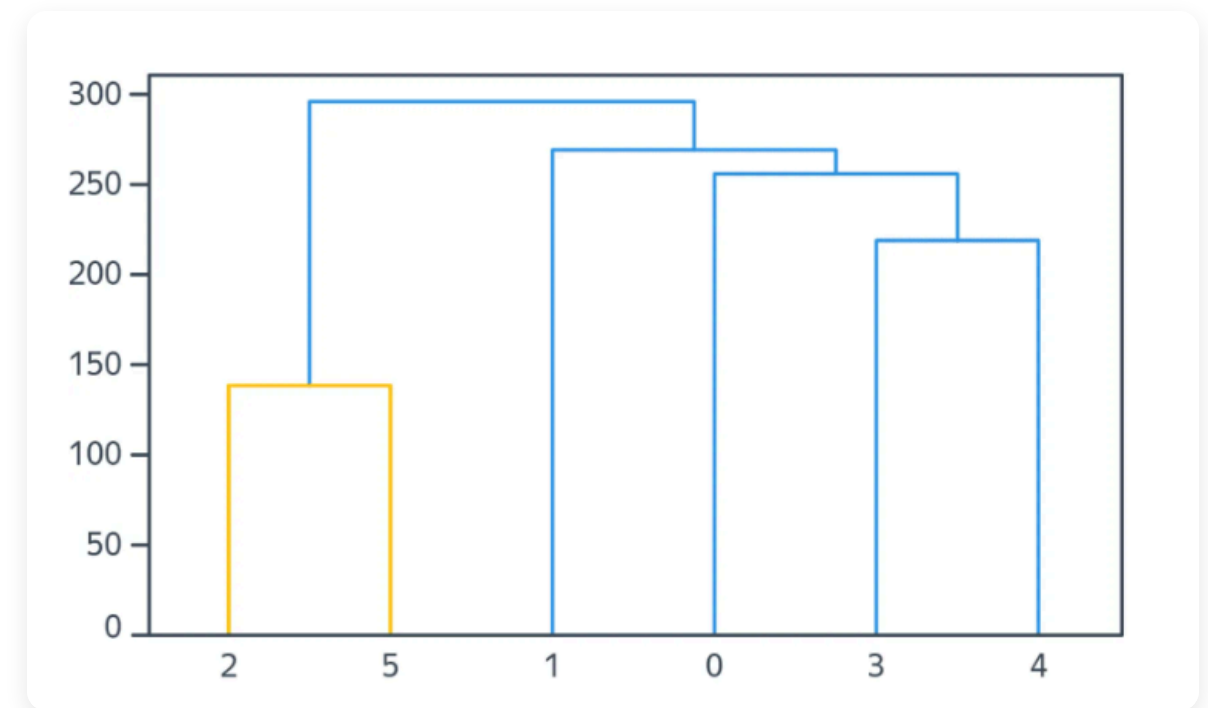




Межкластерные расстояния

Как считать расстояние между кластерами, если там много точек?

- **Single linkage:** минимальное расстояние между точками двух кластеров. Такой метод склонен к образованию цепочек кластеров.
- **Complete linkage:** максимальное расстояние между точками. Приводит к очень компактным кластерам, может разбить один кластер на 2, если в нем есть несколько удаленных точек
- **Average linkage:** среднее всех попарных расстояний.
- **Расстояние Уорда:** объединяются те кластеры, которые минимально увеличивают общую дисперсию.



Дендрограмма позволяет обрезать дерево на любом уровне для выбора нужного K .



Свойства иерархической кластеризации

К преимуществам иерархической кластеризации можно отнести то, что алгоритм не требует заранее задавать итоговое число кластеров. Кроме того, получаемая в процессе работы дендрограмма является очень удобным и наглядным инструментом для визуализации структуры данных.

Главный недостаток метода заключается в его высокой ресурсоёмкости. Алгоритм требует вычисления матрицы попарных расстояний между всеми объектами, что занимает $O(N^2)$ памяти. Вычислительная сложность составляет $O(N^3)$ (или $O(N^2 \log N)$ при эффективной реализации), что делает его неприменимым для больших выборок.

Также стоит учитывать, что алгоритм использует жадную стратегию: если на раннем этапе было принято ошибочное решение об объединении двух кластеров, то в дальнейшем исправить эту ошибку уже невозможно.



Оценка качества: Внутрикластерное расстояние

Показывает, насколько компактными получились кластеры. Чем это значение **меньше**, тем лучше.

$$F_{in} = \frac{1}{K} \sum_{k=1}^K \frac{\sum_{x_i, x_j \in C_k, i < j} \rho(x_i, x_j)}{|C_k|(|C_k| - 1)/2}$$

- C_k — множество объектов k -го кластера, а $|C_k|$ — их количество.
- **Числитель дроби:** сумма расстояний между всеми парами точек внутри одного кластера.
- **Знаменатель дроби:** количество таких пар. То есть сама дробь вычисляет *среднее расстояние внутри конкретного кластера*.
- Внешняя сумма с $\frac{1}{K}$ просто усредняет эти показатели по всем K кластерам.



Оценка качества: Межкластерное расстояние

Показывает, насколько кластеры отделены друг от друга. Чем это значение **больше**, тем лучше.

$$F_{out} = \frac{2}{K(K-1)} \sum_{k < m} \frac{\sum_{x_i \in C_k, x_j \in C_m} \rho(x_i, x_j)}{|C_k| \cdot |C_m|}$$

- Мы берем все пары разных кластеров.
- Каждое слагаемое - это среднее расстояние между всеми точками первого кластера и всеми точками второго.
- Внешняя сумма усредняет эти расстояния по всем парам кластеров.



Оценка качества: Коэффициент Силуэта

Ещё одна метрика, не требующая разметки. Изначально коэффициент определяется для каждого объекта, а метрика для всей выборки вводится как средний коэффициент силуэта для всех объектов.

$$S(x_i) = \frac{B(x_i) - A(x_i)}{\max(B(x_i), A(x_i))}$$

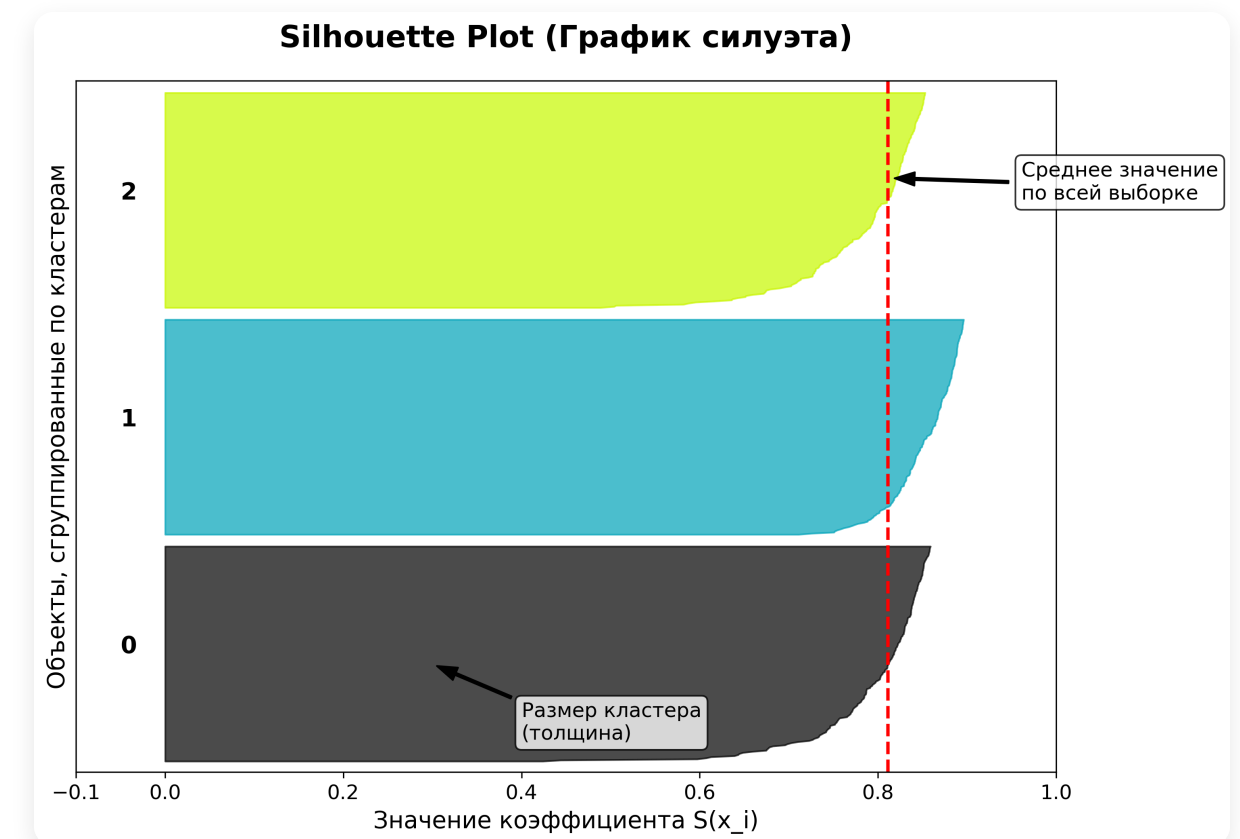
- $A(x_i)$ — среднее расстояние между x_i и объектами своего кластера.
- $B(x_i)$ — среднее расстояние между x_i и объектами ближайшего чужого кластера.
- В идеальном случае объекты своего кластера ближе, то есть $A(x_i) < B(x_i)$. Но если свой кластер имеет вытянутую форму, а рядом есть кластер поменьше, может оказаться, что $A(x_i) > B(x_i)$.
- Сам коэффициент принимает значения **от -1 до +1** и максимизируется, когда кластеры кучные и хорошо отделены друг от друга.



Как читать график силуэта?

Для визуализации качества кластеризации используют *Silhouette plot*.

- **По оси Y:** Все объекты выборки, сгруппированные по кластерам. Ширина полосы (толщина «ножа») показывает количество объектов в кластере.
- **По оси X:** Значение коэффициента $S(x_i)$ для каждого конкретного объекта.
- **Цвета:** Разные кластеры выделены разными цветами для наглядности.
- **Красная пунктирная линия:** Среднее значение силуэта по всей выборке. Это наш главный ориентир.





Подбор K с помощью Силуэта

Анализ силуэта используется для изучения расстояния между полученными кластерами и эффективного выбора оптимального значения K .

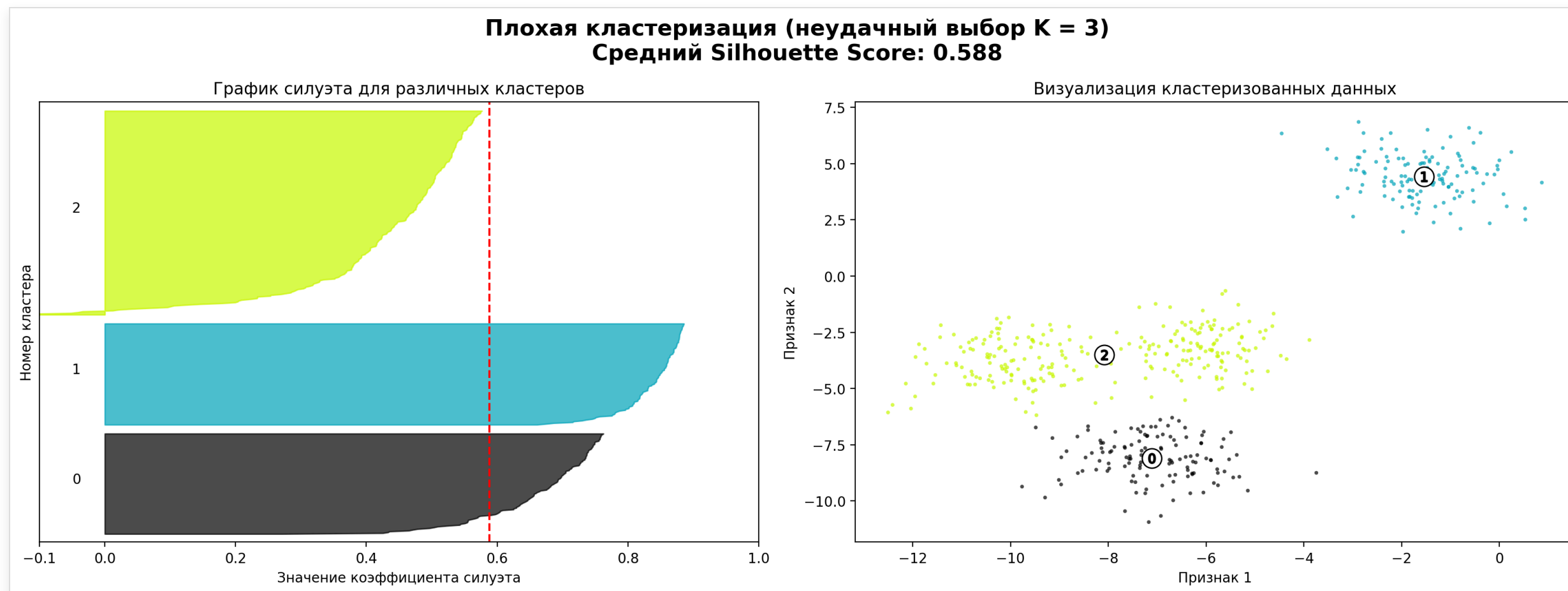
На что смотреть при анализе графика:

- В первую очередь ищем K , при котором общее среднее значение силуэта (красная пунктирная линия) максимально.
- В удачной модели большинство точек каждого кластера должны иметь значение силуэта выше среднего.
- Если кластер содержит много точек с отрицательным силуэтом, это верный признак того, что объекты отнесены не к своему кластеру.
- Оптимальная кластеризация, как правило, дает кластеры сопоставимого размера. Флуктуации толщины говорят о неудачном разбиении.



Пример: Неудачный выбор K

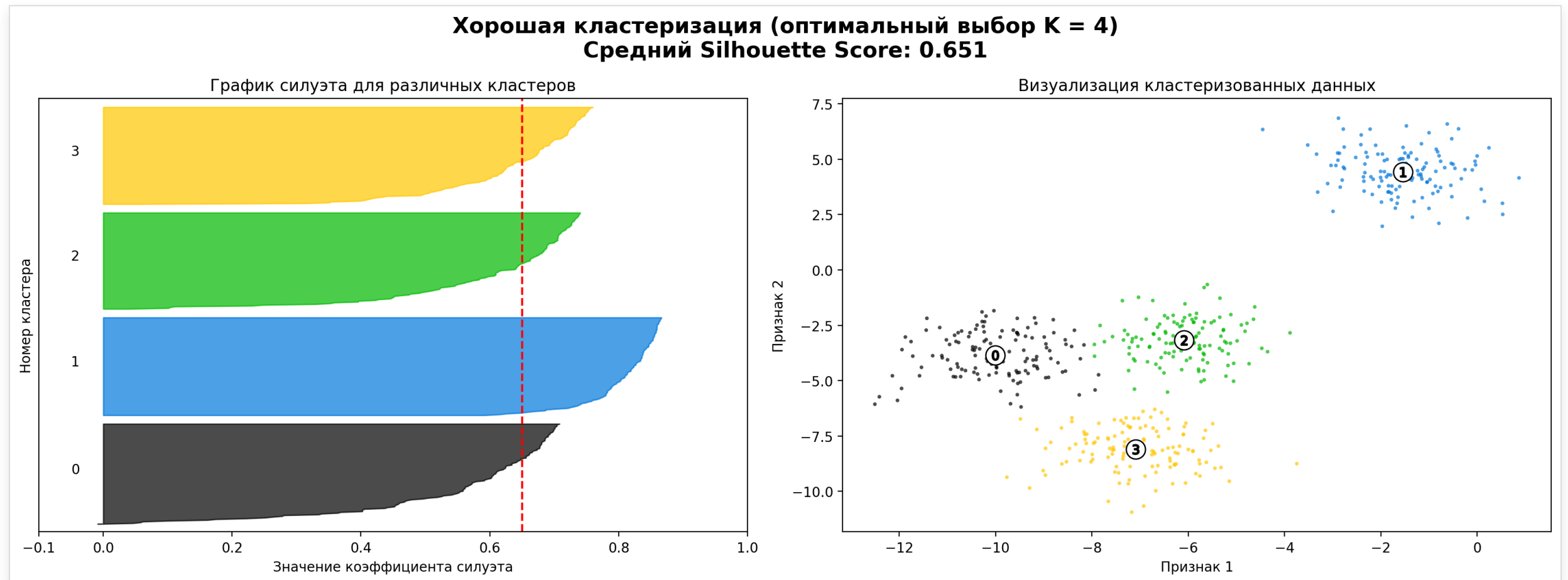
Здесь мы видим явные проблемы: один кластер (№2) получился огромным, а его значения силуэта сильно не дотягивают до среднего. Толщина "ножей" крайне неравномерна.





Пример: Оптимальный выбор K

Все кластеры имеют схожую толщину, значения силуэта в основном положительны, и в каждом кластере количество объектов с силуэтом выше среднего достаточно велико.





Машинное обучение

Спасибо за внимание!

Вопросы?

Докладчик: Ваня Горохов